

Búsqueda, ordenación y evaluación experimental

TC-CH06 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Ordenar un millón de registros cuesta un trabajo que no había que hacer para responder una sola pregunta. ¿Cuándo compensa pagarlo por adelantado, y cuándo es tirar el tiempo?

Producto final. Un conteo a mano de las comparaciones de la ordenación por selección, contrastado con la medición, y un veredicto razonado sobre el escalado real al duplicar la entrada.

Mapa del tema

1. Las dos operaciones más fundamentales	1
2. Algoritmos de Búsqueda	2
3. Algoritmos de Ordenación	4
4. Límite inferior teórico de la ordenación por comparación	7
5. Criterios de Selección y Evaluación Experimental	9
6. La operación peligrosa: recursión profunda en QuickSort	9
7. Actividad de depuración: fallos en partición y búsqueda	10
8. Síntesis operativa y tablas de diagnóstico	10
9. Glosario conceptual	11
10. Conexión con el curso y siguientes pasos	11
11. Fuentes y reproducibilidad	11

Cómo usar este tema

Buscar y ordenar son las dos operaciones que aparecen debajo de casi todo lo demás, y este capítulo las trata juntas porque están relacionadas: ordenar es caro, y solo compensa si después se va a buscar muchas veces. Esa es la decisión que el capítulo enseña a tomar con números en la mano.

Al terminar sabrá: contar las comparaciones de un algoritmo de ordenación y comprobar su cuenta midiendo; explicar por qué ningún algoritmo basado en comparaciones puede bajar de cierto límite, y por qué la ordenación por dígitos sí puede; decidir si conviene ordenar antes de buscar, según cuántas búsquedas vayan a hacerse; y reconocer qué significa que una ordenación sea estable, y cuándo eso importa con datos biológicos.

La ruta **esencial** son las dos búsquedas, las ordenaciones cuadráticas y eficientes, y el criterio de cuándo ordenar. La ruta de **estudio** añade la demostración del límite inferior, la ordenación por dígitos, la evaluación experimental y el anexo.

1 Las dos operaciones más fundamentales

ESENCIAL En cualquier pipeline de análisis biomédico —desde la identificación de variantes en secuenciación masiva (NGS) hasta la anotación funcional de transcritos—, dos operaciones se repiten millones de veces por segundo: **buscar** un elemento específico y **ordenar** una colección masiva de registros.

Ambas operaciones sirven como el banco de pruebas empírico definitivo para los paradigmas algorítmicos del capítulo de coste y diseño:

- La fuerza bruta se plasma en la **búsqueda lineal** y las ordenaciones elementales cuadráticas.
- El paradigma Divide y Vencerás da lugar a la **búsqueda binaria**, al **MergeSort** y al **QuickSort**.
- El uso de estructuras jerárquicas avanzadas (el montículo del capítulo de estructuras no lineales) origina el **HeapSort**.

Además, en este capítulo alcanzaremos uno de los resultados más profundos de la informática teórica: el **límite inferior de la ordenación por comparación** ($\Omega(n \log n)$), demostrando por qué ningún algoritmo basado en comparar pares de elementos, lo escriba una persona o una máquina, puede bajar de ese límite, que no es una conjetura sino un resultado demostrado, y cómo los algoritmos de distribución como **Radix Sort** logran sortearla mediante propiedades específicas de las secuencias biológicas.

Buscar y ordenar son las dos operaciones más frecuentes sobre cualquier colección de datos; sirven de banco de pruebas de los paradigmas de diseño ya vistos, fuerza bruta, divide y vencerás y uso de una estructura—aplicados al mismo problema con costes muy distintos.

2 Algoritmos de Búsqueda

2.1 Búsqueda lineal (Fuerza bruta)

Concepto. Antes de entrar en las dos búsquedas de este capítulo conviene situarlas. Las dos son *ciegas* o no informadas: no saben nada del problema salvo cómo comparar dos elementos. La lineal no supone nada; la binaria supone que los datos están ordenados. Hay otras dos familias que aquí no desarrollamos. Las búsquedas *informadas* usan una estimación del problema para decidir por dónde seguir, y de esa familia viene la heurística de semillas de BLAST que verá en el capítulo de alineamiento. Las búsquedas *con adversario*, como el minimax con poda, suponen que alguien juega en contra, y quedan fuera del curso. Saber que existen evita creer que toda búsqueda se reduce a recorrer o a partir por la mitad.

La **búsqueda lineal** examina secuencialmente cada elemento de la colección desde el índice 0 hasta encontrar el objetivo o agotar los n elementos. Su ventaja reside en su universalidad: no exige que los datos guarden ningún orden previo.

La búsqueda lineal recorre la colección comparando cada elemento con el buscado; su coste es $O(n)$ en el peor caso y no exige que la colección esté ordenada.

Análisis formal del caso promedio.

Si el elemento buscado está presente y tiene idéntica probabilidad ($1/n$) de ocupar cualquiera de las n posiciones, el número medio de comparaciones es la media aritmética de la serie $1, 2, 3, \dots, n$:

$$\text{Promedio} = \frac{1}{n} \sum_{i=1}^n i = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Al descartar constantes en notación asintótica, el caso promedio sigue siendo $O(n)$. Para $n = 100$ elementos, el número medio de comparaciones es exactamente 50.5.

Si el elemento buscado está presente y tiene la misma probabilidad de estar en cualquiera de las n posiciones, el número promedio de comparaciones de la búsqueda lineal es $(1 + 2 + \dots + n)/n = (n+1)/2$ —la media de la serie de costes $1, 2, \dots, n$ de encontrarlo en cada posición—. En notación O la constante $1/2$ se descarta y el caso promedio sigue siendo $O(n)$: promediar no cambia la clase de complejidad. La ventaja de la búsqueda lineal es no imponer precondition alguna sobre el orden de los datos; su desventaja es no escalar a colecciones grandes.

Simulando la búsqueda lineal del elemento situado en cada una de las $n = 100$ posiciones de una colección, la suma real de comparaciones necesarias es 5050 y el promedio es 50.5 —exactamente $(n+1)/2 = 50.5$ —, confirmando por ejecución directa, no solo por fórmula, el caso promedio de la búsqueda lineal.

Se reproduce con `verification/chapter_results.sh linear_search_average 100`.

2.2 Búsqueda binaria (Divide y vencerás)

Sobre una colección previamente ordenada, la **búsqueda binaria** compara el objetivo con el elemento central:

- Si coincide, termina con éxito.
- Si el objetivo es menor, descarta la mitad derecha.
- Si el objetivo es mayor, descarta la mitad izquierda.

La búsqueda binaria descarta la mitad de los datos restantes en cada comparación, logrando $O(\log n)$; a cambio exige que la colección esté ordenada de antemano.

Deducción de la relación de recurrencia.

Sea $T(n)$ el tiempo de ejecución sobre una entrada de tamaño n . En cada paso se realiza una comparación (c) y el tamaño se reduce a la mitad:

$$T(n) = T(n/2) + c$$

Tras k pasos sucesivos, el tamaño remanente es $n/2^k$. El proceso se detiene cuando queda 1 elemento ($n/2^k = 1 \Rightarrow 2^k = n$), lo que da estrictamente $k = \log_2(n)$ pasos.

El coste $O(\log n)$ de la búsqueda binaria se deriva de la recurrencia $T(n) = T(n/2) + c$ —cada paso cuesta una comparación constante c y reduce el tamaño del problema a la mitad—: tras k pasos el tamaño restante es $n/2^k$, y el algoritmo termina cuando $n/2^k = 1$, es decir, $k = \log_2(n)$.

Por ejemplo, sobre el array ordenado $[2, 3, 5, 7, 11, 13, 17, 19]$ de 8 elementos, buscar el valor 11 requiere exactamente $\log_2(8) = 3$ pasos:

1. Paso 1: medio en índice 3 ($A[3] = 7 < 11$), descarta mitad izquierda.
2. Paso 2: medio en índice 5 ($A[5] = 13 > 11$), descarta mitad derecha.
3. Paso 3: medio en índice 4 ($A[4] = 11$), objetivo localizado.

Sobre el array ordenado $2, 3, 5, 7, 11, 13, 17, 19$ (8 elementos), buscar 11 con búsqueda binaria tarda exactamente 3 pasos— $\log_2(8) = 3$, coincidiendo con la predicción de la recurrencia— y lo encuentra en el índice 4: paso 1 descarta la mitad izquierda comparando con 7, paso 2 descarta la mitad derecha comparando con 13, y paso 3 compara con 11 y lo halla.

Se reproduce con `verification/chapter_results.sh binary_search_trace 100 37`.

```
ALGORITMO: Busqueda_Binaria_Iterativa
ENTRADA:
- A: Array ordenado de n elementos A[0..n-1] con precondition A[0] <= A[1] <= ... <= A[n-1]
- objetivo: Valor buscado
SALIDA:
- Indice: Posicion k tal que A[k] == objetivo, o -1 si no existe en A
COMPLEJIDAD: Tiempo  $O(\log n)$  en el peor caso, Espacio auxiliar  $O(1)$ 

1. INICIALIZACION DE LIMITES:
  izq <- 0
  der <- n - 1

2. BUCLE DICOTOMICO:
  Mientras izq <= der:
    # Calculo seguro de la posicion central evitando desbordamiento de enteros
    medio <- izq + (der - izq) / 2

    Si A[medio] == objetivo entonces:
      Retornar medio # Elemento localizado con exito
    Sino si A[medio] < objetivo entonces:
      izq <- medio + 1 # Descartar subvector izquierdo A[izq..medio]
    Sino:
      der <- medio - 1 # Descartar subvector derecho A[medio..der]

3. TERMINACION:
  Retornar -1 # Rango agotado sin coincidencias
```

El error de aplicar búsqueda binaria sobre datos no ordenados.

Si los datos no están estrictamente ordenados, la búsqueda binaria descarta mitades asumiendo una invariante inexistente, perdiendo elementos presentes y generando falsos negativos críticos.

Aplicar búsqueda binaria sobre un array sin ordenar no garantiza encontrar un valor presente: las decisiones izquierda/derecha asumen un orden inexistente y pueden descartar la mitad que contiene el objetivo.

Se reproduce con `verification/chapter_results.sh binary_search_unsorted`.

2.3 La búsqueda con centinela

ESTUDIO Una optimización clásica de la búsqueda lineal consiste en escribir el valor buscado al final de la colección, como *centinela*. Así el bucle no necesita comprobar en cada vuelta si ha llegado al final: sabe que va a encontrar el valor, y solo al terminar mira si lo encontró en la posición del centinela, en cuyo caso no estaba. Se ahorra una comparación por vuelta, que es la mitad del trabajo del bucle.

Concepto. El coste sigue siendo $O(n)$. La optimización cambia la constante, no la clase. Es un buen recordatorio de que una mejora real de rendimiento puede no aparecer en la notación asintótica, y también de lo contrario: que dos algoritmos de la misma clase pueden tardar el doble uno que otro.

3 Algoritmos de Ordenación

Ordenar una secuencia es el prerequisite computacional que desbloquea la búsqueda binaria, la detección inmediata de duplicados en lecturas NGS y el cálculo de percentiles de expresión génica.

Ordenar permite la búsqueda binaria (de $O(n)$ a $O(\log n)$), convierte preguntas caras sobre datos sin ordenar —mediana, duplicados, percentiles— en triviales sobre datos ordenados, y es una precondition que muchos algoritmos posteriores asumen sin comprobar.

3.1 Algoritmos elementales cuadráticos ($O(n^2)$)

1. Bubble Sort (Ordenación por burbuja).

Compara pares de elementos adyacentes y los intercambia si están desordenados. En cada pasada completa, el mayor elemento remanente «burbujea» hasta su posición final a la derecha. Con optimización de parada temprana (detectar si una pasada no realizó ningún intercambio), logra un mejor caso $O(n)$ sobre listas previamente ordenadas.

Bubble sort compara pares adyacentes y los intercambia si están desordenados, haciendo que el elemento mayor remanente burbujee a su sitio en cada pasada; con la optimización de detectar una pasada sin intercambios se detiene antes ante una entrada favorable. En una lista ya ordenada de tamaño n esa detección da un mejor caso $O(n)$ (una pasada, $n - 1$ comparaciones, cero intercambios); en una invertida se mantiene en $O(n^2)$ ($(n - 1) + (n - 2) + \dots + 1$ comparaciones en total repartidas en hasta n pasadas, la última solo para confirmar que ya no hay intercambios).

Bubble sort sobre $[5, 2, 8, 1, 3]$ produce pasada a pasada: $[2, 5, 1, 3, 8]$ tras la primera (el 8 llega al final), $[2, 1, 3, 5, 8]$ tras la segunda, y $[1, 2, 3, 5, 8]$ tras la tercera —ya ordenada—, confirmada por una cuarta pasada sin intercambios.

Se reproduce con `verification/chapter_results.sh bubble_sort_trace`.

2. Insertion Sort (Ordenación por inserción).

Construye la lista ordenada de forma incremental de izquierda a derecha. Para cada elemento, lo desplaza hacia la izquierda dentro del bloque ya ordenado hasta encontrar su posición correcta. Es muy adaptativo: sobre datos casi ordenados toma tiempo lineal $O(n)$, y tiene una sobrecarga de ejecución tan baja que es el método más rápido en la práctica para arrays pequeños, típicamente por debajo de unos 15 elementos.

Insertion sort desplaza cada elemento hacia la izquierda dentro de la parte ya ordenada hasta encontrar su hueco; en una lista ya ordenada de tamaño n no se desplaza ningún elemento —mejor caso $O(n)$ —, mientras que en una lista invertida cada elemento se desplaza hasta el principio con $(n - 1) + (n - 2) + \dots + 1$ desplazamientos en total —peor caso $O(n^2)$ —: el mismo comportamiento adaptativo que bubble sort pero con menos trabajo por comparación, lo que lo hace más rápido en la práctica pese a compartir la misma cota asintótica.

3. Selection Sort (Ordenación por selección).

Divide el array en una sección ordenada y otra no ordenada. En cada pasada, localiza el mínimo absoluto del bloque desordenado y lo intercambia con la primera posición libre. Realiza siempre exactamente $n(n - 1)/2$ comparaciones, por lo que su coste es **estrictamente** $O(n^2)$ **en todos los casos** (no posee mecanismo de parada temprana). Su única ventaja es que realiza como máximo $n - 1$ intercambios físicos en memoria, siendo preferible

si la escritura es extremadamente costosa frente a la comparación.

Selection sort busca el mínimo de la parte sin ordenar en cada una de las n pasadas: $O(n^2)$ en todos los casos, no solo en el peor. Etiquetarlo como $O(n \log n)$ confunde un método simple con uno eficiente.

Se reproduce con `verification/chapter_results.sh selection_sort_comparisons 8`.

Selection sort realiza a lo sumo un intercambio por pasada, así que sobre una lista de tamaño n nunca hace más de $n - 1$ intercambios en total, muy por debajo de sus $O(n^2)$ comparaciones; es preferible cuando la operación de escribir/intercambiar es costosa y la de comparar barata, a diferencia de bubble e insertion sort, que pueden intercambiar en casi cada comparación.

Selection sort sobre $[5, 2, 8, 1, 3]$ produce pasada a pasada: $[1, 2, 8, 5, 3]$ tras la primera (intercambia 5 con el mínimo 1), $[1, 2, 8, 5, 3]$ tras la segunda (2 ya está en su sitio, sin intercambio), y $[1, 2, 3, 5, 8]$ tras la tercera (intercambia 8 con el mínimo remanente 3), ya ordenada con 2 intercambios efectivos en total frente a los intercambios repartidos en 3 pasadas de bubble sort. A diferencia de bubble sort, selection sort ejecuta de todos modos una cuarta pasada ($i = 3$, comparando 5 y 8) aunque el array ya esté ordenado porque no tiene mecanismo de parada temprana, razón de que sea $O(n^2)$ en todos los casos y no solo en el peor.

Se reproduce con `verification/chapter_results.sh selection_sort_trace`.

3.2 Algoritmos eficientes ($O(n \log n)$)

1. MergeSort (Ordenación por mezcla).

Aplica Divide y Vencerás de forma canónica: divide la lista recursivamente en dos mitades hasta alcanzar sublistas de longitud 1 (que están trivialmente ordenadas), y luego las recombina mediante una rutina de mezcla lineal ($O(n)$). Su recurrencia $T(n) = 2T(n/2) + O(n)$ garantiza un rendimiento estricto de $O(n \log n)$ en todos los casos (peor, mejor y promedio). Es además un algoritmo **estable**, pero requiere un array auxiliar de memoria $O(n)$.

Merge sort divide en $\log n$ niveles y hace un trabajo de mezcla $O(n)$ en cada nivel: $\log n$ veces $O(n)$ da $O(n \log n)$ garantizado en todos los casos, a cambio de memoria extra para la mezcla.

Mezclar dos sublistas ya ordenadas de longitudes a y b compara siempre las cabezas de ambas, mueve la menor al resultado y avanza en esa sublista hasta vaciar una de las dos —momento en que el resto de la otra se copia sin más comparaciones por estar ya ordenada—; mezclar $[1, 3, 5]$ con $[2, 4, 6]$ produce $[1, 2, 3, 4, 5, 6]$ en exactamente 5 comparaciones, una menos que la suma de longitudes (6) porque el último elemento remanente no necesita comparación.

Se reproduce con `verification/chapter_results.sh merge_two_sorted_halves`.

2. QuickSort (Ordenación rápida por partición).

Selecciona un elemento como **pivote** y reordena el array en tiempo lineal de forma que todos los elementos menores que el pivote queden a su izquierda y los mayores a su derecha, ordenando luego recursivamente cada partición. En el caso promedio es extraordinariamente rápido ($O(n \log n)$) y opera *in-place* sin requerir memoria auxiliar para los datos. Sin embargo, si el pivote elegido es sistemáticamente el mínimo o máximo sobre datos ya ordenados, el árbol de recursión degenera a altura n , provocando un peor caso de $O(n^2)$.

Con un quicksort que elige siempre el primer elemento como pivote, ejecutarlo sobre una lista ya ordenada produce en cada partición un lado vacío y casi todo el resto del otro: n niveles de partición cada uno $O(n)$, total $O(n^2)$, el peor caso y no el $O(n \log n)$ promedio.

Se reproduce con `verification/chapter_results.sh quicksort_fixed_pivot_sorted 8`.

Para mitigar este riesgo, los motores modernos emplean la **mediana de tres** (tomar como pivote la mediana entre el primer, medio y último elemento) y hacen una transición a **Insertion Sort** cuando los subvectores tienen menos de 15 elementos.

Elegir la mediana de tres elementos (primero, medio, último) como pivote mitiga el peor caso de quicksort sobre datos ya ordenados sin el coste $O(n)$ de calcular la mediana real —que garantizaría peor caso $O(n \log n)$ pero penalizaría el promedio—; una segunda mitigación habitual e independiente del pivote es cambiar a insertion

sort para subarrays pequeños, por debajo de unos 15 elementos, cuya menor sobrecarga por elemento lo hace más rápido que seguir recursando quicksort en entradas diminutas.

```
ALGORITMO: QuickSort_Particion_Hoare
ENTRADA:
- A: Array de elementos mutables A[izq..der]
SALIDA:
- Array A reordenado in-situ con todos los elementos <= pivote a la izquierda y >= pivote a la derecha
COMPLEJIDAD: Tiempo  $O(\text{der} - \text{izq} + 1)$  lineal por pasada, Espacio auxiliar  $O(1)$  in-place

FUNCION Particionar_Hoare(A, izq, der):
    pivote <- A[izq]          # Seleccion de pivote (o mediana de 3)
    i <- izq - 1
    j <- der + 1

    Bucle Infinito:
        Repetir i <- i + 1 hasta que A[i] >= pivote
        Repetir j <- j - 1 hasta que A[j] <= pivote

        Si i >= j entonces:
            Retornar j        # j es el punto de corte para la subdivision recursiva

        Intercambiar(A[i], A[j])

FUNCION QuickSort_Recursivo(A, izq, der):
    Si izq < der entonces:
        corte <- Particionar(A, izq, der)
        QuickSort_Recursivo(A, izq, corte)
        QuickSort_Recursivo(A, corte + 1, der)
```

3. HeapSort (Ordenación por montículo).

Aprovecha el montículo de máximos del capítulo de estructuras no lineales. Transforma el array en un heap en tiempo $O(n)$ y luego extrae sucesivamente el elemento máximo colocándolo al final del array. Realiza n extracciones de coste $O(\log n)$, garantizando $O(n \log n)$ estricto *in-place*. Sin embargo, no es estable y su recorrido de memoria no contiguo presenta peor localidad espacial de caché que QuickSort.

Heapsort inserta los n elementos en un heap y los extrae uno a uno: como cada extracción del máximo cuesta $O(\log n)$, ordenar n elementos con un heap cuesta $O(n \log n)$.

3.3 Estabilidad y espacio en memoria

Un algoritmo de ordenación es **estable** si preserva estrictamente el orden de llegada relativo entre registros que comparten la misma clave. Por ejemplo, si se ordenan lecturas de secuenciación con idéntica calidad Phred ($Q = 90$), un algoritmo estable garantiza que la lectura que entró primero (L_1) permanezca antes que la que entró después (L_3).

Ordenando las lecturas $(L_1, 90)$, $(L_2, 85)$, $(L_3, 90)$ por puntuación decreciente (L_1 llegó antes que L_3 , ambas con puntuación 90), un método estable produce siempre $(L_1, 90)$, $(L_3, 90)$, $(L_2, 85)$ —conservando el orden de llegada en los empates—; un método inestable puede producir $(L_3, 90)$, $(L_1, 90)$, $(L_2, 85)$, con la misma puntuación correctamente ordenada pero sin garantía sobre el orden relativo de L_1 y L_3 . La estabilidad solo se manifiesta cuando hay claves repetidas.

Se reproduce con `verification/chapter_results.sh stability_demo`.

Algoritmo	Peor Caso	Caso Promedio	Memoria Extra	¿Estable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	No
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	Sí
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	Sí
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n)$ auxiliar	Sí
QuickSort	$O(n^2)$	$O(n \log n)$	$O(\log n)$ prom. / $O(n)$ peor	No
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(1)$ in-place	No

Nota metodológica sobre las implementaciones analizadas: En QuickSort la memoria de pila es $O(\log n)$ bajo particiones equilibradas y puede degenerar a $O(n)$ en el peor caso sin optimización; la técnica de recurrir sobre la partición menor (Sección 6) garantiza una profundidad máxima de pila $O(\log n)$. Entre los métodos de esta tabla, ninguno reúne simultáneamente cota estricta $O(n \log n)$, estabilidad y memoria auxiliar $O(1)$.

Merge sort es estable pero usa memoria extra; quicksort es in-place pero no estable; heapsort es in-place y tampoco estable. Ningún método eficiente combina las tres ventajas ($O(n \log n)$ garantizado, estable, in-place) a la vez.

4 Límite inferior teórico de la ordenación por comparación

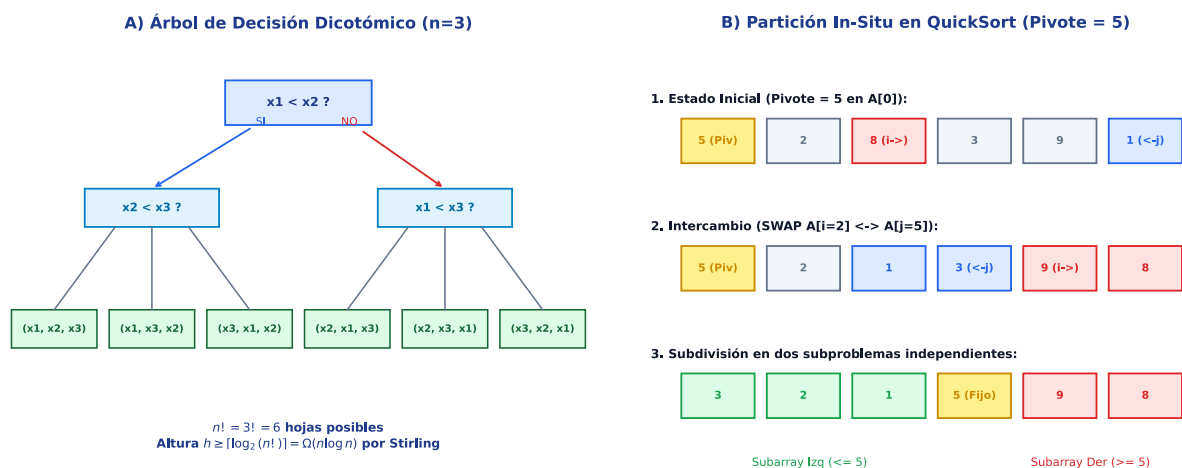


Figura 1: A) Árbol de decisión dicotómico para $n = 3$, ilustrando por qué ordenar por comparación exige una altura mínima de $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ hojas posibles. B) Traza in-situ del esquema de partición de QuickSort (Hoare) con punteros convergentes i y j alrededor del pivote. Figura original adaptada de las presentaciones docentes.

¿Es posible diseñar un algoritmo de ordenación por comparación que tome tiempo lineal $O(n)$ en el peor caso? La respuesta matemática es un rotundo **NO**.

Demostración por Árbol de Decisión.

Cualquier algoritmo que ordene mediante comparaciones binarias ($a_i < a_j$) se puede modelar como un árbol de decisión donde:

- Cada nodo interno representa una comparación con dos salidas posibles (Sí / No).
- Cada hoja representa una de las permutaciones finales posibles del array.
- Para n elementos distintos, existen exactamente $n!$ permutaciones posibles, por lo que el árbol debe contener al menos $L \geq n!$ hojas.

Un árbol binario de altura h tiene como máximo 2^h hojas. Por tanto:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

Para acotar ese logaritmo basta un argumento elemental, sin necesidad de la fórmula de Stirling. La suma tiene n términos crecientes; quedándonos solo con la mitad superior, cada uno de ellos vale al menos $\log_2(n/2)$:

$$\log_2(n!) = \sum_{i=1}^n \log_2(i) \geq \sum_{i=n/2}^n \log_2(n/2) = \frac{n}{2} \log_2(n/2) = \Omega(n \log n)$$

Dado que la altura del árbol representa el número de comparaciones en el peor caso, ningún algoritmo de comparación puede requerir menos de $\Omega(n \log n)$ pasos.

Ningún algoritmo que ordene comparando elementos puede requerir menos de $\Omega(n \log n)$ comparaciones en el peor caso: existen $n!$ permutaciones posibles y cada comparación binaria solo distingue dos casos, exigiendo un árbol de decisión de altura mínima $\lceil \log_2(n!) \rceil = \Omega(n \log n)$.

4.1 Ordenación no comparativa: Radix Sort para secuencias biológicas

El límite $\Omega(n \log n)$ aplica **exclusivamente a algoritmos basados en comparaciones**. Si las claves poseen estructura interna conocida —como secuencias de ADN de longitud fija k sobre el alfabeto $\Sigma = \{A, C, G, T\}$ —, se puede ordenar mediante **LSD Radix Sort** (ordenación por dígitos menos significativos).

Radix Sort distribuye las claves en cubetas según el símbolo en la posición i (desde el extremo derecho k hasta el 1) de forma estable. Realiza k pasadas de coste lineal $O(n)$:

$$\text{Coste total} = O(k \cdot n)$$

Para ordenar 10^9 lecturas de k -meros con $k = 8$, Radix Sort toma 8×10^9 operaciones elementales, superando ampliamente a cualquier método de comparación ($10^9 \times \log_2(10^9) \approx 30 \times 10^9$ operaciones).

El límite $\Omega(n \log n)$ aplica exclusivamente a algoritmos por comparación; Radix Sort ordena símbolo a símbolo sin comparar claves completas, alcanzando un coste asintótico $O(k \cdot n)$ con k la longitud fija de la clave. Para secuencias de ADN cortas y alfabeto acotado, realiza k pasadas lineales de distribución estable.

LSD radix sort ordena de derecha a izquierda: cada pasada distribuye las claves en cubetas según el símbolo en la posición actual y las recoge en orden de cubeta manteniendo el orden relativo dentro de cada una (distribución estable, imprescindible para que el resultado final quede ordenado). Sobre GAT, ACT, GAC, ACA ($k = 3$, alfabeto A, C, G, T), 3 pasadas —una por posición de última a primera— producen ACA, ACT, GAC, GAT, ordenadas alfabéticamente sin comparar nunca dos claves completas entre sí.

Se reproduce con `verification/chapter_results.sh radix_sort_dna`.

```
ALGORITMO: Radix_Sort_Secuencias_ADN
ENTRADA:
- Secuencias: Lista de n k-meros de longitud fija k sobre alfabeto Sigma = {'A', 'C', 'G', 'T'}
SALIDA:
- Secuencias_Ordenadas: Lista ordenada lexicograficamente en tiempo lineal
COMPLEJIDAD: Tiempo  $O(k * (n + |\Sigma|))$ , Espacio auxiliar  $O(n + |\Sigma|)$ 

1. BUCLE DE PASADAS POSICIONALES (De derecha a izquierda, d = k-1 descendiendo a 0):
  Para d desde k - 1 descendiendo hasta 0:
    # Inicializar cubetas estables para cada base del alfabeto
    Cubeta['A'] <- Lista_Vacia()
    Cubeta['C'] <- Lista_Vacia()
    Cubeta['G'] <- Lista_Vacia()
    Cubeta['T'] <- Lista_Vacia()

    # Fase 1: Distribucion estable en cubetas segun el caracter en posicion d
    Para cada secuencia S en Secuencias:
      base <- S[d]
      Cubeta[base].ENQUEUE(S) # Preserva el orden relativo previo (estabilidad)

    # Fase 2: Recoleccion concatenada en orden canonico A -> C -> G -> T
```



```
Secuencias <- Concatenar(Cubeta['A'], Cubeta['C'], Cubeta['G'], Cubeta['T'])
```

```
2. RETORNO:  
Retornar Secuencias
```

5 Criterios de Selección y Evaluación Experimental

No existe un algoritmo universalmente superior; la elección depende del patrón de acceso del pipeline biomédico:

No hay un algoritmo de ordenación universalmente mejor: la elección depende del tamaño de la entrada, de si se necesita garantía en el peor caso, y de si las claves tienen estructura (longitud fija) que un método no comparativo pueda aprovechar.

5.1 ¿Cuándo compensa ordenar antes de buscar?

Si sobre una colección desordenada de n elementos se van a realizar S búsquedas:

- **Opción A (Búsqueda lineal directa):** coste total = $S \cdot O(n)$.
- **Opción B (Ordenar primero y luego búsqueda binaria):** coste total = $O(n \log n) + S \cdot O(\log n)$.

Para una única búsqueda ($S = 1$), ordenar previamente es un desperdicio computacional ($n \log n + \log n > n$). Solo compensa ordenar cuando el coste del ordenamiento se **amortiza** entre un número suficiente de consultas:

$$S > \frac{n \log_2 n}{n - \log_2 n} \approx \log_2 n$$

Para una sola búsqueda sobre datos sin ordenar, ordenar primero y buscar con búsqueda binaria cuesta más que una búsqueda lineal directa; ordenar antes de buscar solo compensa cuando se va a buscar muchas veces sobre la misma colección, porque el coste de ordenar se amortiza entre las búsquedas.

Se reproduce con `verification/chapter_results.sh sort_once_vs_many_searches 1000 100`.

5.2 Evaluación experimental: escalado ante duplicación (2×)

Protocolo de benchmark y reproducibilidad experimental.

Para contrastar empíricamente las tasas teóricas de crecimiento, el protocolo debe controlar estrictamente las variables experimentales: fijar la semilla aleatoria ($SEED=42$), generar la misma distribución de claves, registrar el número exacto de comparaciones (métrica determinista e invariable frente a la máquina) y promediar múltiples ejecuciones para amortizar variaciones del planificador del sistema operativo.

Al duplicar el tamaño de la entrada ($n \rightarrow 2n$):

- En algoritmos $O(n^2)$ (Selection Sort), el tiempo se multiplica por $(2n)^2/n^2 = 4\times$.
- En algoritmos $O(n \log n)$ (MergeSort), el tiempo se multiplica suavemente por $\frac{2n \log(2n)}{n \log n} = 2 \left(1 + \frac{1}{\log n}\right) \approx 2.2\times$.

Duplicando el tamaño de la entrada, el tiempo real de selection sort escala aproximadamente $\times 4$ (compatible con $O(n^2)$) y el de merge sort escala de forma mucho más suave (compatible con $O(n \log n)$): la teoría predice la tendencia, la medición confirma que se cumple en esta máquina y con estos datos.

Se reproduce con `verification/chapter_results.sh time_compare_sorts 200`.

6 La operación peligrosa: recursión profunda en QuickSort

Un fallo crítico en entornos de producción ocurre cuando QuickSort procesa listas de gran tamaño que ya vienen casi ordenadas y el pivote se elige de forma ingenua (primer elemento). La profundidad del árbol de llamadas alcanza n , provocando un **desbordamiento de la pila de llamadas** (*Stack Overflow*) del sistema operativo.

Protocolo preventivo en 3 pasos:

1. Utilizar siempre la regla de pivote mediana-de-tres o pivote aleatorio.
2. Establecer un umbral de corte para delegar en la ordenación por inserción por debajo de unos 15 elementos. La cifra es un valor típico medido en bibliotecas reales, no una constante universal: depende de la máquina y del tipo de dato, y se ajusta midiendo.
3. Recurrir en primer lugar sobre la partición menor e iterar sobre la mayor para acotar el uso de pila a $O(\log n)$.

7 Actividad de depuración: fallos en partición y búsqueda

Existen dos esquemas de partición clásicos. El de **Hoare**, que aparece más arriba, hace converger dos punteros desde los extremos. El de **Lomuto**, que se depura a continuación, recorre el array con un solo puntero y coloca el pivote en su sitio al final. Los dos son correctos cuando están bien escritos; el de abajo no lo está.

Analice este código con tres fallos:

```
particionar(arr, bajo, alto):
    pivote <- arr[bajo]
    i <- bajo
    # Fallo 1: el limite del bucle no incluye el extremo derecho (alto)
    para j de bajo + 1 hasta alto - 1:
        # Fallo 2: comparacion incorrecta que invierte el orden del particionado
        si arr[j] > pivote:
            i <- i + 1
            intercambiar(arr[i], arr[j])
    # Fallo 3: se intercambia con la posicion original del pivote en vez de con i
    intercambiar(arr[bajo], arr[alto])
    devolver i
```

8 Síntesis operativa y tablas de diagnóstico

8.1 Tabla de diagnóstico de síntomas computacionales

Síntoma observado	Causa probable	Comprobación y corrección
Búsqueda binaria no encuentra elemento presente	El array no estaba previamente ordenado	Asegurar ordenación estricta antes de invocar
QuickSort tarda $O(n^2)$ en datos reales	Peor caso por pivote extremo en datos ordenados	Implementar mediana-de-tres o pivote aleatorio
El orden relativo de registros idénticos cambia	Uso de un algoritmo inestable (QuickSort/HeapSort)	Cambiar a MergeSort o incluir índice como clave secundaria
Tiempo de ejecución se cuadruplica al duplicar n	Comportamiento cuadrático $O(n^2)$ activo	Migrar a un algoritmo $O(n \log n)$ o Radix Sort

8.2 Tabla de síntesis con preguntas de control

Requisito del pipeline	Algoritmo recomendado	Pregunta de control
Una sola búsqueda puntual	Búsqueda lineal $O(n)$	¿Se realizarán más búsquedas posteriores?
Búsquedas masivas frecuentes	Ordenar una vez + Búsqueda binaria	¿El volumen de búsquedas amortiza el ordenamiento?
Ordenación estable garantizada	MergeSort	¿Se dispone de memoria auxiliar $O(n)$?

Requisito del pipeline	Algoritmo recomendado	Pregunta de control
Máxima velocidad <i>in-place</i>	QuickSort (con mediana-de-tres)	¿Se ha prevenido el desbordamiento de pila?
Secuencias cortas de ADN (k -meros)	Radix Sort ($O(n \cdot k)$)	¿La longitud de las secuencias es constante?

9 Glosario conceptual

REFERENCIA Use este glosario para consultar propiedades de búsqueda, ordenación y complejidad sin interrumpir el recorrido principal.

- **Búsqueda lineal:** escaneo secuencial exhaustivo en $O(n)$.
- **Búsqueda binaria:** descarte recursivo por mitades en $O(\log n)$.
- **Estabilidad:** preservación del orden relativo original ante claves iguales.
- **In-place:** algoritmo que opera sin requerir memoria auxiliar proporcional a n .
- **Pivote:** elemento de referencia para la partición en QuickSort.
- **MergeSort:** algoritmo Divide y Vencerás estable con coste $O(n \log n)$ garantizado.
- **HeapSort:** ordenación mediante montículo binario en $O(n \log n)$ *in-place*.
- **Stirling:** aproximación matemática $\ln(n!) \approx n \ln n - n$.
- **Radix Sort:** ordenación no comparativa por cubetas en $O(n \cdot k)$.
- **Amortización:** reparto del coste de una operación previa entre múltiples usos posteriores.

10 Conexión con el curso y siguientes pasos

La ordenación y la indexación sostienen el análisis de frecuencias e información del capítulo siguiente, los modelos probabilísticos del posterior, el filtrado por semillas de BLAST en el de alineamiento y el aprendizaje automático del último.

- El capítulo siguiente aplica conteo ordenado de frecuencias y perfiles de motivos con matrices de pesos.
- El de modelos probabilísticos usa ordenación de estados y caminos.
- El de alineamiento apoya en la ordenación su filtrado por semillas.
- El de aprendizaje automático ordena y particiona conjuntos de datos.

11 Fuentes y reproducibilidad

Este capítulo se apoya en las dos presentaciones docentes de algoritmos de la asignatura, escritas por Álvaro Serrano Navarro. La demostración del límite inferior, los esquemas de partición y el análisis de las ordenaciones siguen a Cormen y colaboradores [3]; la ordenación por dígitos aplicada a secuencias y el orden de magnitud de los datos de secuenciación, a Compeau y Pevzner [2]; y el protocolo de medición con semilla fija, a Allesina y Wilmes [1].

Todas las cuentas y mediciones se reproducen con `verification/chapter_results.sh` tal como están impresas.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos contrastivos de búsqueda y ordenación; no sostiene ninguna afirmación en solitario.
- [2] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático de la ordenación de k -mers con radix sort y su papel en la indexación de BLAST.

- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para la prueba del límite de la ordenación por comparación y el análisis de radix sort; no se reproduce, solo se cita.